Machine Learning 2 (APM_50441_EP/DSAIB - 2026-2027). Lab6

The objectives of the lab

This lab illustrates the sixth lecture (*Reinforcement Learning*) on the Cliff Walking problem of the lecture. Its goals are:

- to describe a Markov decision process and its model, and to solve it by dynamic programming (policy evaluation, value iteration, policy iteration),
- to compare Monte Carlo and temporal difference estimation of a value function,
- to implement SARSA and Q-learning, and to understand why they learn different paths,
- to implement the REINFORCE policy gradient in PyTorch, and to see the effect of the reward-to-go on its variance.

The optional *To go further* part, after the core, can be skipped at first: it covers bandits, more temporal difference methods, the deadly triad, deep RL (DQN, actor-critic, PPO) and the RL of language models (GRPO, DPO, agents).

Use of AI. AI assistants (e.g. the ones built into Colab or VS Code) are allowed, but you must be able to explain every line you use, and answer the *predict before you run* questions before running the code.

Core (about 2 hours)

Exercise 1 (The environment)

Cliff Walking (Sutton and Barto, Example 6.6, and the lecture) is a 4×12 grid: the agent starts in S (bottom left), must reach G (bottom right), and pays -1 per step; stepping into the cliff costs -100 and sends it back to S. The states are numbered $s = 12 \times \text{row} + \text{column}$ and the actions are $0 =$ up, $1 =$ right, $2 =$ down, $3 =$ left. A `gymnasium` environment is used through `reset()` and `step(action)`, which returns the next state, the reward, and whether the episode is *terminated* (goal reached) or *truncated* (time limit).

1. *Predict before you run:* what is the average return of the uniformly random policy, with episodes cut after 500 steps? Run 200 such episodes in `RL.make_env(max_steps=500)` and report the mean return, the mean length and the proportion of episodes reaching the goal.

   The lecture also uses a *slippery* version: with probability 10%, the chosen action is replaced by a uniformly random one. `RL.tabular_model(slip)` returns its model: the transition probabilities `P[s, a, s']` and the expected rewards `R[s, a]` (the goal is absorbing, with reward 0).

2. Build the model with `slip=0.1`. Check that each `P[s, a]` is a probability distribution. Where can the agent go from S when it chooses *right*, with which probabilities? Compute `R[START, RIGHT]` by hand and compare.

Exercise 2 (Dynamic programming)

With a known model and a discount $\gamma = 0.9$, the value of a stationary policy π solves the Bellman equation $v_\pi = r_\pi + \gamma P_\pi v_\pi$, with $P_\pi(s, s') = \sum_a \pi(a|s) P(s, a, s')$ and $r_\pi(s) = \sum_a \pi(a|s) R(s, a)$. It can be computed exactly by solving this linear system, or by iterating the Bellman operator $v \mapsto r_\pi + \gamma P_\pi v$, a γ-contraction for the sup norm (PC6). The optimal value $v_\star$ is the fixed point of the *optimal* Bellman operator $v \mapsto \max_a \big(R(\cdot, a) + \gamma P(\cdot, a) v\big)$.

1. For the uniform policy on the slippery grid ($\gamma = 0.9$), compute v_π exactly with `np.linalg.solve`, then by 60 iterations of the Bellman operator from $v = 0$. Plot the sup-norm error (over the valid states) along the iterations in log scale, with the bound $\gamma^k \|v_0 - v_\pi\|_\infty$. Display v_π on the grid.

2. Implement value iteration (iterate the optimal Bellman operator from $v = 0$ until the sup-norm change is below 10^{-10}). Display $v_\star$ with the greedy policy $\arg\max_a \left(R(s,a) + \gamma P(s,a) v_\star\right)$ and its path from S (without slips). Then solve the problem without slips (`slip=0`) and compare the paths.

3. Implement policy iteration: start from the policy "always right" (which walks into the cliff), evaluate it exactly, take the greedy policy, and repeat until the policy does not change. How many iterations are needed? Plot the sup-norm error $\|v_{\pi_k} - v_\star\|_\infty$ along the iterations together with the one of value iteration.

Exercise 3 (**Monte Carlo and temporal difference estimation**)

Without a model, the value of a policy is estimated from episodes. *Monte Carlo* (MC) moves $V(s_t)$ towards the observed return G_t; *temporal difference* TD(0) moves it towards the *bootstrapped* target $r_{t+1} + \gamma V(s_{t+1})$, after each step. We compare them on the random walk of the lecture (Sutton and Barto, Example 6.2): 5 states A to E, start in C, left or right with probability $1/2$, reward 1 when leaving on the right; the true values are $1/6, \dots, 5/6$. `RL.random_walk_episode(rng)` returns the transitions (s, r, s') of an episode (`$s' = $ None` at the end).

1. Implement every-visit MC and TD(0) with a constant step size α and the values initialized at 0.5 (no discount). For $\alpha \in \{0.01, 0.02, 0.05, 0.1, 0.2, 0.3\}$, compute the RMS error over the 5 states after each of 100 episodes, averaged over 100 runs. Plot the curves of the best α of each method, and the final error (mean over the last 10 episodes) as a function of α.

2. With a constant step size, the estimates keep fluctuating. Compare, over 1000 episodes (20 runs), MC and TD(0) with $\alpha = 0.05$ and with the decreasing step size $\alpha_n = 1/n$ (n = number of updates of the state). Which ones converge to the true values? (PC6, Exercise 2.)

Exercise 4 (**SARSA and Q-learning**)

Without a model, control learns the action values $Q(s,a)$ from the transitions, while acting ε-greedily. After a transition (s, a, r, s') and the choice of the next action a':

- SARSA (on-policy) uses the target $r + \gamma Q(s', a')$: it learns the value of the ε-greedy policy it follows,
- Q-learning (off-policy) uses the target $r + \gamma \max_b Q(s', b)$: it learns the value of the greedy policy.

As in the lecture, we use $\varepsilon = 0.1$, $\alpha = 0.5$, no discount, and no slip.

- *Predict before you run:* which method will collect more reward during learning? Implement both on `RL.make_env(max_steps=1000)` (500 episodes, ties broken at random, Q initialized at 0), average the reward per episode over 20 runs, and plot it (smoothed over 10 episodes). Draw the greedy paths of the learned Q (averaged over the runs).

Exercise 5 (**Policy gradient: REINFORCE in PyTorch**)

A policy gradient method optimizes a parametrized policy directly. With a tabular softmax policy $\pi_\theta(a|s) \propto e^{\theta_{s,a}}$, the REINFORCE estimator of $\nabla_\theta J(\theta)$ for an episode $(s_0, a_0, r_1, \dots)$ is $\sum_t G \, \nabla_\theta \log \pi_\theta(a_t|s_t)$, with either the total return $G = G_0$ or, using causality, the *reward-to-go* $G = G_t = \sum_{t' \geq t} r_{t'+1}$ (PC6). In PyTorch, it is the gradient of the loss $-\sum_t G \log \pi_\theta(a_t|s_t)$, computed by automatic differentiation.

To keep the episodes short, they start from a random valid state and are cut after 100 steps (no discount, no slip). The quality of a policy is measured exactly with the model: `expected_return` computes the expected sum of rewards over 100 steps from S (backward induction).

1. Write a function sampling an episode with the policy softmax(θ) (return the states, the actions and the rewards), and the REINFORCE loss with the full return or the reward-to-go. Train θ (initialized at 0, plain SGD with step sizes $3 \cdot 10^{-4}$ for the full return and 10^{-3} for the reward-to-go) for 4000 episodes, 3 runs each, and plot `expected_return` every 100 episodes. The optimum is -13.

2. At the initial policy ($\theta = 0$), sample 2000 episodes and compute, for each of them, the coordinate $\partial / \partial \theta_{S,\text{up}}$ of the gradient of the REINFORCE objective (minus the gradient of the loss) with both estimators. Compare their means and standard deviations. Why is the reward-to-go legitimate, and why does it help?

To go further

The following blocks are optional and independent: each of them only uses the objects defined in the core part. Their difficulty is marked by stars: $\star$ direct extension, $\star\star$ new method, $\star\star\star$ longer or research-flavored.

Block	Lecture	Stars
Multi-armed bandits	Bandits	$\star\star$
Monte Carlo control	Monte Carlo control	$\star$
n-step TD	n-step methods and eligibility traces	$\star\star$
Maximization bias and double Q-learning	Q-learning, DQN overestimation	$\star\star$
Transfer to Taxi	Tabular methods	$\star$
Baird's counterexample	The deadly triad	$\star\star$
Baseline and actor-critic	Policy gradient, actor-critic	$\star\star$
GRPO: group-relative advantages	LLMs: reasoning with verifiable rewards	$\star\star$
DQN on CartPole	Deep Q-networks	$\star\star$
PPO	Proximal policy optimization	$\star\star$
Preferences: reward model and DPO	RLHF and DPO	$\star\star$
A tiny agent with retrieval	The agent loop, RAG	$\star\star\star$

Exercise 6 (**Multi-armed bandits** $\star\star$)
A bandit is an MDP with a single state: at each step, the agent pulls one of K arms and receives a noisy reward. The *regret* $\sum_t(\mu_\star - \mu_{A_t})$ measures the cost of learning. ε-greedy explores at a constant rate (linear regret), UCB plays $\arg\max_a \hat{\mu}_a + \sqrt{2\log t/N_a}$ (optimism in the face of uncertainty) and Thompson sampling plays the best arm of a sample of the posterior.

- On the 10-armed Gaussian testbed (means $\mu_a \sim \mathcal{N}(0,1)$, unit noise, horizon 2000, 100 runs), compare the cumulative regret of ε-greedy ($\varepsilon = 0.1$), UCB and Thompson sampling (prior $\mathcal{N}(0,1)$ on each mean, known unit noise).

Exercise 7 (**Monte Carlo control** $\star$)
Monte Carlo control alternates the evaluation of Q by averaging the returns of complete episodes and an ε-greedy improvement. It needs no bootstrapping, but each update waits for the end of the episode.

- Implement on-policy every-visit MC control on Cliff Walking ($\varepsilon = 0.1$, episodes from S cut after 200 steps, 2000 episodes), with $Q(s,a)$ updated towards the observed returns either by their average (step size $1/n$) or with a constant step size $\alpha = 0.1$. Which paths are learned? Compare with SARSA and Q-learning.

Exercise 8 (n-**step TD** $\star\star$)
n-step TD uses the target $r_{t+1} + \cdots + r_{t+n} + V(s_{t+n})$: $n = 1$ is TD(0), $n = \infty$ is MC. Intermediate values often work best (Sutton and Barto, Figure 7.2).

- On the random walk with 19 states (`RL.random_walk_episode(rng, n=19)`, true values $(s+1)/20$), compute the RMS error over the first 10 episodes (averaged over 100 runs, values initialized at 0.5) of n-step TD for $n \in \{1, 2, 4, 8, 16, 64\}$ as a function of α.

Exercise 9 (**Maximization bias and double Q-learning** $\star\star$)
The target $\max_b Q(s', b)$ of Q-learning is a maximum of noisy estimates, hence biased upwards: $\mathbb{E}[\max_b \hat{Q}(b)] \geq \max_b \mathbb{E}[\hat{Q}(b)]$. Double Q-learning keeps two tables, selects the action with one and evaluates it with the other.

In the small MDP of Sutton and Barto (Example 6.7), from A the action *right* ends the episode with reward 0, while *left* leads to B, from which 10 actions end the episode with a reward $\mathcal{N}(-0.1, 1)$: going left is a mistake.

- Run Q-learning and double Q-learning ($\varepsilon = 0.1$, $\alpha = 0.1$, no discount, 300 episodes, 1000 runs) on this MDP, and plot the proportion of *left* actions in A along the episodes. Check first the bias numerically: the mean of the maximum of 10 sample means of 5 draws of $\mathcal{N}(-0.1, 1)$.

Exercise 10 (Transfer to Taxi ⋆)

`Taxi-v3` is another tabular environment: 500 states (taxi position, passenger location, destination) and 6 actions (moves, pick up, drop off), reward -1 per step, $+20$ for a correct drop-off and -10 for an illegal pick-up or drop-off.

- Adapt your Q-learning to `gym.make('Taxi-v3')` ($\varepsilon = 0.1$, $\alpha = 0.5$, $\gamma = 0.99$, 3000 episodes), plot the reward per episode, and evaluate the greedy policy on 100 new episodes.

Exercise 11 (Baird's counterexample ⋆⋆)

The *deadly triad* (function approximation, bootstrapping and off-policy learning) can make TD diverge. Baird's counterexample (Sutton and Barto, Example 11.1): 7 states, all rewards 0 (so $v_\pi = 0$), $\gamma = 0.99$; the behavior policy goes to one of the 6 upper states with probability $6/7$ and to state 7 with probability $1/7$, the target policy always goes to state 7 (importance ratio 7 or 0). The linear features are $v(i) = 2w_i + w_8$ for $i \leq 6$ and $v(7) = w_7 + 2w_8$: $w = 0$ is a perfect fit.

- Run off-policy semi-gradient TD(0) ($\alpha = 0.01$, $w_0 = (1, 1, 1, 1, 1, 1, 10, 1)$, 1000 steps) with these linear features and with tabular features (same initial values), and plot the largest $|v(s)|$ along the steps. Then run the linear version *on-policy* (transitions drawn from the target policy… and from the behavior policy, without importance ratio).

Exercise 12 (Baseline and actor-critic ⋆⋆)

Subtracting a baseline $b(s_t)$ from the reward-to-go does not bias the policy gradient (PC6) and reduces its variance; with a learned value $V(s_t)$ as baseline, the advantage $G_t - V(s_t)$ measures whether the action was better than usual. An *actor-critic* replaces G_t by the TD target $r_{t+1} + \gamma V(s_{t+1})$ and updates after each step.

- With $\gamma = 0.99$, compare on Cliff Walking (2000 episodes, 3 runs, same episodes as in the core part) the reward-to-go REINFORCE, REINFORCE with a tabular value baseline (step size 0.1 for V), and the one-step actor-critic (critic step size 0.1). Measure the policies with the exact discounted value from S.

Exercise 13 (GRPO: group-relative advantages ⋆⋆)

To fine-tune language models with verifiable rewards, GRPO (DeepSeek, 2024) avoids learning a critic: it samples a *group* of G answers to the same prompt and uses as advantage the reward normalized within the group, $A_i = (r_i - \bar{r})/s_r$. RLOO uses $r_i - \frac{1}{G-1}\sum_{j \neq i} r_j$, an unbiased leave-one-out baseline. The same idea applies to our episodes: the "prompt" is the start state, the "answer" an episode.

- Train the tabular softmax policy on Cliff Walking with groups of $G = 8$ episodes from the same random start state (each episode weighted by its group advantage, the same for all its steps), with the GRPO and the RLOO advantages, and with the raw return (the full-return REINFORCE of the core part), for 1000 groups (8000 episodes, 2 runs).

Exercise 14 (DQN on CartPole ⋆⋆)

With a large or continuous state space, Q is approximated by a neural network. DQN (Mnih et al., 2015) stabilizes the Q-learning of a network with an *experience replay* buffer (decorrelated mini-batches of past transitions) and a *target network* (a frozen copy, updated periodically, to compute the targets). `CartPole-v1`: keep a pole balanced on a cart (state of dimension 4, 2 actions, reward $+1$ per step, at most 500 steps).

- Implement DQN (network $4 \to 64 \to 64 \to 2$, Adam with learning rate 10^{-3}, buffer of $10\,000$ transitions, mini-batches of 64, $\gamma = 0.99$, ε decreasing linearly from 1 to 0.05 over $10\,000$ steps, target network updated every 500 steps) for $30\,000$ steps, and plot the length of the episodes.

Exercise 15 (PPO ⋆⋆)

PPO (Schulman et al., 2017) reuses each batch of episodes for several gradient steps, with the *clipped* objective $\mathbb{E}\big[\min(\rho A, \mathsf{clip}(\rho, 1-\epsilon, 1+\epsilon)A)\big]$, $\rho = \pi_\theta(a|s)/\pi_{\mathsf{old}}(a|s)$: the policy has no incentive to move the probability ratio beyond $1 \pm \epsilon$. It is the algorithm of RLHF (Lecture 6), and a standard choice in control.

- Plot the clipped objective as a function of ρ for $A = 1$ and $A = -1$ ($\epsilon = 0.2$). Then train the PPO of `stable-baselines3` on `CartPole-v1` for $30\,000$ steps and evaluate it on 10 episodes.

Exercise 16 (Preferences: reward model and DPO ⋆⋆)

RLHF learns from comparisons: a human prefers answer y_1 to y_2 with probability $\sigma(r(y_1) - r(y_2))$ (Bradley–Terry). The classical pipeline fits a reward model on the preferences, then maximizes $\mathbb{E}_\pi[r] - \beta\,\mathsf{KL}(\pi\|\pi_{\mathsf{ref}})$, whose solution is $\pi_\star \propto \pi_{\mathsf{ref}} e^{r/\beta}$ (PC6). DPO (Rafailov et al., 2023) plugs this solution into the Bradley–Terry likelihood and fits the policy *directly* on the preferences: $-\log\sigma\big(\beta \log \frac{\pi(y_w)}{\pi_{\mathsf{ref}}(y_w)} - \beta \log \frac{\pi(y_l)}{\pi_{\mathsf{ref}}(y_l)}\big)$.

- Take a single prompt with 20 possible answers, true rewards $r \sim \mathcal{N}(0,1)$, a uniform reference policy and $\beta = 0.5$. Simulate 2000 preferences between random pairs of answers. Fit (i) a reward model by logistic regression on the pairs, then the KL-regularized policy; (ii) the DPO policy by gradient descent on its loss (logits of a softmax policy). Compare both with $\pi_\star$ (total variation distance) and their expected true rewards.

Exercise 17 (A tiny agent with retrieval ⋆⋆⋆)

An *agent* is a language model in a loop: at each step it can call a tool (a search engine, a calculator…), read the result, and decide what to do next, until it answers (lecture: *The Agent Loop*). *Retrieval-augmented generation* (RAG) puts the documents retrieved for the question in the prompt. We use a small open model (Qwen2.5-0.5B-Instruct, about 1 GB, downloaded once; a GPU runtime on Colab is faster, and a laptop CPU takes a few minutes) with two tools: a retriever over a few notes on this lab (BM25-like scores) and a calculator. Small models are unreliable agents: the point is to see the loop, and its failures.

- Run the agent loop below on the three questions. For each step, look at the model's output, the tool call and the observation. Then estimate the cost of an agent: if each step reads a context of 3000 tokens and writes 200 tokens, with prices of $1\$$ per million input tokens and $4\$$ per million output tokens, how much does a 10-step task cost? If each step succeeds independently with probability 0.95, what is the probability that the 10 steps all succeed?

Use of AI. Artificial intelligence tools have been used to revise the content of this lab. The main contributor takes full responsibility for this content.

License. The texts of this lab are under the Creative Commons Attribution-ShareAlike 4.0 license (CC BY-SA 4.0), and its code under the MIT license; the datasets keep their own licenses. Main contributor: E. Le Pennec. Contributors: S. Boucheron, A. Dieuleveut, A.K. Fermin, S. Gadat, S. Gaiffas, B. Goujaud, A. Guilloux, Ch. Keribin, Th. Klein, E. Matzner, J. Read, M. Sangnier, E. Scornet.